

Program Analysis 2025-26

Lecture #6

The Real IL



The Real Incorrectness Logic (IL)

IL, as you have seen it

Incorrectness Logic

PETER W. O’HEARN, Facebook and University College London, UK

Program correctness and incorrectness are two sides of the same coin. As a programmer, even if you would like to have correctness, you might find yourself spending most of your time reasoning about incorrectness. This includes informal reasoning that people do while looking at or thinking about their code, as well as that supported by automated testing and static analysis tools. This paper describes a simple logic for program incorrectness which is, in a sense, the other side of the coin to Hoare’s logic of correctness.

CCS Concepts: • **Theory of computation** → **Programming logic**.

Additional Key Words and Phrases: Proofs, Bugs, Static Analysis

ACM Reference Format:

Peter W. O’Hearn. 2020. Incorrectness Logic. *Proc. ACM Program. Lang.* 4, POPL, Article 10 (January 2020), 32 pages. <https://doi.org/10.1145/3371078>

1 INTRODUCTION

When reasoning informally about a program, people make abstract inferences about what might go wrong, as well as about what must go right. A programmer might ask “will the program crash if we give it a large string?”, without saying *which* large string. In this paper we investigate the hypothesis that reasoning about the presence of bugs can be underpinned by sound techniques in a principled logical system, just as reasoning about correctness (absence of bugs) has been demonstrated to have sound logical principles in an extensive research literature. We also consider the relationship of the principles to automated reasoning tools for finding bugs in software.

We explore our hypothesis by defining incorrectness logic, a formalism that is similar to Hoare’s logic of program correctness [Hoare 1969], except that it is oriented to proving incorrectness rather than correctness. Hoare’s theory is based on specifications of the form

$\{pre-condition\}code\{post-condition\}$

which say that the post-condition *over-approximates* (describes a superset of) the states reachable upon termination when the code is executed starting from states satisfying the pre-condition (the so-called strongest post). Conversely, we use a specification form

$[presumption]code[result]$

which says that the post-assertion *result* be an under-approximation (subset) of the final states that can be reached starting from states satisfying the *presumption*.

The under-approximate triples were studied (with a different but equivalent definition) previously by de Vries and Koutavas [2011] in their reverse Hoare logic, which they used to specify randomized algorithms. Incorrectness logic adds post-assertions for errors as well as for normal termination, and these assertions describe erroneous states that can be reached by actual program executions. Dijkstra [1976] famously remarked that “testing can be quite effective for showing the presence of bugs, but is hopelessly inadequate for showing their absence,” and he made this remark while arguing for the

Author’s address: Peter W. O’Hearn, Facebook and University College London, UK.



This work is licensed under a Creative Commons Attribution 4.0 International License.

© 2020 Copyright held by the owner/author(s).

2475-1421/2020/1-ART10

<https://doi.org/10.1145/3371078>

Proc. ACM Program. Lang., Vol. 4, No. POPL, Article 10. Publication date: January 2020.

10

$[P] \ c \ [Q]$



The real IL

Incorrectness Logic

PETER W. O’HEARN, Facebook and University College London, UK

Program correctness and incorrectness are two sides of the same coin. As a programmer, even if you would like to have correctness, you might find yourself spending most of your time reasoning about incorrectness. This includes informal reasoning that people do while looking at or thinking about their code, as well as that supported by automated testing and static analysis tools. This paper describes a simple logic for program incorrectness which is, in a sense, the other side of the coin to Hoare’s logic of correctness.

CCS Concepts: • **Theory of computation** → **Programming logic**.

Additional Key Words and Phrases: Proofs, Bugs, Static Analysis

ACM Reference Format:

Peter W. O’Hearn. 2020. Incorrectness Logic. *Proc. ACM Program. Lang.* 4, POPL, Article 10 (January 2020), 32 pages. <https://doi.org/10.1145/3371078>

1 INTRODUCTION

When reasoning informally about a program, people make abstract inferences about what might go wrong, as well as about what must go right. A programmer might ask “will the program crash if we give it a large string?”, without saying *which* large string. In this paper we investigate the hypothesis that reasoning about the presence of bugs can be underpinned by sound techniques in a principled logical system, just as reasoning about correctness (absence of bugs) has been demonstrated to have sound logical principles in an extensive research literature. We also consider the relationship of the principles to automated reasoning tools for finding bugs in software.

We explore our hypothesis by defining incorrectness logic, a formalism that is similar to Hoare’s logic of program correctness [Hoare 1969], except that it is oriented to proving incorrectness rather than correctness. Hoare’s theory is based on specifications of the form

$\{pre\text{-}condition\}code\{post\text{-}condition\}$

which say that the post-condition *over-approximates* (describes a superset of) the states reachable upon termination when the code is executed starting from states satisfying the pre-condition (the so-called strongest post). Conversely, we use a specification form

$[presumption]code[result]$

which says that the post-assertion *result* be an under-approximation (subset) of the final states that can be reached starting from states satisfying the *presumption*.

The under-approximate triples were studied (with a different but equivalent definition) previously by de Vries and Koutavas [2011] in their reverse Hoare logic, which they used to specify randomized algorithms. Incorrectness logic adds post-assertions for errors as well as for normal termination, and these assertions describe erroneous states that can be reached by actual program executions. Dijkstra [1976] famously remarked that “testing can be quite effective for showing the presence of bugs, but is hopelessly inadequate for showing their absence,” and he made this remark while arguing for the

Author’s address: Peter W. O’Hearn, Facebook and University College London, UK.



This work is licensed under a Creative Commons Attribution 4.0 International License.

© 2020 Copyright held by the owner/author(s).

2475-1421/2020/1-ART10

<https://doi.org/10.1145/3371078>

Proc. ACM Program. Lang., Vol. 4, No. POPL, Article 10. Publication date: January 2020.

10

$\epsilon \in \{ok, er\}$

$[P] c [\epsilon : Q]$



The real IL

Incorrectness Logic

PETER W. O’HEARN, Facebook and University College London, UK

Program correctness and incorrectness are two sides of the same coin. As a programmer, even if you would like to have correctness, you might find yourself spending most of your time reasoning about incorrectness. This includes informal reasoning that people do while looking at or thinking about their code, as well as that supported by automated testing and static analysis tools. This paper describes a simple logic for program incorrectness which is, in a sense, the other side of the coin to Hoare’s logic of correctness.

CCS Concepts: • **Theory of computation** → **Programming logic**.

Additional Key Words and Phrases: Proofs, Bugs, Static Analysis

ACM Reference Format:

Peter W. O’Hearn. 2020. Incorrectness Logic. *Proc. ACM Program. Lang.* 4, POPL, Article 10 (January 2020), 32 pages. <https://doi.org/10.1145/3371078>

1 INTRODUCTION

When reasoning informally about a program, people make abstract inferences about what might go wrong, as well as about what must go right. A programmer might ask “will the program crash if we give it a large string?”, without saying *which* large string. In this paper we investigate the hypothesis that reasoning about the presence of bugs can be underpinned by sound techniques in a principled logical system, just as reasoning about correctness (absence of bugs) has been demonstrated to have sound logical principles in an extensive research literature. We also consider the relationship of the principles to automated reasoning tools for finding bugs in software.

We explore our hypothesis by defining incorrectness logic, a formalism that is similar to Hoare’s logic of program correctness [Hoare 1969], except that it is oriented to proving incorrectness rather than correctness. Hoare’s theory is based on specifications of the form

$\{pre\text{-}condition\}code\{post\text{-}condition\}$

which say that the post-condition *over-approximates* (describes a superset of) the states reachable upon termination when the code is executed starting from states satisfying the pre-condition (the so-called strongest post). Conversely, we use a specification form

$[presumption]code[result]$

which says that the post-assertion *result* be an under-approximation (subset) of the final states that can be reached starting from states satisfying the *presumption*.

The under-approximate triples were studied (with a different but equivalent definition) previously by de Vries and Koutavas [2011] in their reverse Hoare logic, which they used to specify randomized algorithms. Incorrectness logic adds post-assertions for errors as well as for normal termination, and these assertions describe erroneous states that can be reached by actual program executions. Dijkstra [1976] famously remarked that “testing can be quite effective for showing the presence of bugs, but is hopelessly inadequate for showing their absence,” and he made this remark while arguing for the

Author’s address: Peter W. O’Hearn, Facebook and University College London, UK.



This work is licensed under a Creative Commons Attribution 4.0 International License.

© 2020 Copyright held by the owner/author(s).

2475-1421/2020/1-ART10

<https://doi.org/10.1145/3371078>

Proc. ACM Program. Lang., Vol. 4, No. POPL, Article 10. Publication date: January 2020.

10

Assignment

$$[p]x = e[ok: \exists x'. p[x'/x] \wedge x = e[x'/x]][er: false]$$

Constancy

$$\frac{[p]C[\epsilon: q]}{[p \wedge f]C[\epsilon: q \wedge f]} \quad Mod(C) \cap Free(f) = \emptyset$$

Substitution I

$$\frac{[p]C[\epsilon: q]}{([p]C[\epsilon: q])(e/x)} \quad (Free(e) \cup \{x\}) \cap Free(C) = \emptyset$$

Empty under-approximates

$$[p]C[\epsilon: false]$$

Unit

$$[p]skip[ok: p][er: false]$$

Iterate zero

$$[p]C^\star[ok: p]$$

Choice (where $i = 1$ or 2)

$$\frac{[p]C_i[\epsilon: q]}{[p]C_1 + C_2[\epsilon: q]} \quad 5$$

Nondet Assignment

$$[p]x = nondet()[ok: \exists x' p][er: false]$$

Local Variable

$$\frac{[p]C(y/x)[\epsilon: q]}{[p]local\ x.C[\epsilon: \exists y.q]} \quad y \notin Free(p, C)$$

Substitution II

$$\frac{[p]C[\epsilon: q]}{([p]C[\epsilon: q])(y/x)} \quad y \notin Free(p, C, q)$$

Consequence

$$\frac{p' \Leftarrow p \quad [p]C[\epsilon: q] \quad q \Leftarrow q'}{[p']C[\epsilon: q']}$$

Sequencing (short-circuit)

$$\frac{[p]C_1[er: r]}{[p]C_1; C_2[er: r]}$$

Iterate non-zero

$$\frac{[p]C^\star; C[\epsilon: q]}{[p]C^\star[\epsilon: q]}$$

Error

$$[p]error()[ok: false][er: p]$$

Disjunction

$$\frac{[p_1]C[\epsilon: q_1] \quad [p_2]C[\epsilon: q_2]}{[p_1 \vee p_2]C[\epsilon: q_1 \vee q_2]}$$

Sequencing (normal)

$$\frac{[p]C_1[ok: q] \quad [q]C_2[\epsilon: r]}{[p]C_1; C_2[\epsilon: r]}$$

Backwards Variant (where n fresh)

$$\frac{[p(n) \wedge nat(n)]C[ok: p(n+1) \wedge nat(n)]}{[p(0)]C^\star[ok: \exists n. p(n) \wedge nat(n)]}$$

Assume

$$[p]assume\ B[ok: p \wedge B][er: false]$$

Exit condition

$$[P] \ c \ [\epsilon : Q]$$

ϵ ranges over exit conditions

ok: normal (successful) execution

er: erroneous execution

$$[y = v] \ x := y \ [ok : x = y = v] \qquad [y = v] \ \text{error}(\) \ [er : y = v]$$

Relational semantics

$$\llbracket \text{skip} \rrbracket_{\text{ok}} \triangleq \{(\sigma, \sigma) \mid \sigma \in \Sigma\}$$

$$\llbracket \text{skip} \rrbracket_{\text{er}} \triangleq \emptyset$$

$$\llbracket b? \rrbracket_{\text{ok}} \triangleq \{(\sigma, \sigma) \mid \sigma \models b\}$$

$$\llbracket b? \rrbracket_{\text{er}} \triangleq \emptyset$$

$$\llbracket x := a \rrbracket_{\text{ok}} \triangleq \{(\sigma, \sigma[\llbracket a \rrbracket \sigma / x]) \mid \sigma \in \Sigma\}$$

$$\llbracket x := a \rrbracket_{\text{er}} \triangleq \emptyset$$

$$\llbracket \text{error}() \rrbracket_{\text{ok}} \triangleq \emptyset$$

$$\llbracket \text{error}() \rrbracket_{\text{er}} \triangleq \{(\sigma, \sigma) \mid \sigma \in \Sigma\}$$

Main
novelty!

$$\llbracket x := \text{nondet}() \rrbracket_{\text{ok}} \triangleq \{(\sigma, \sigma[x \mapsto v]) \mid \sigma \in \Sigma, v \in \mathbb{Z}\}$$

$$\llbracket x := \text{nondet}() \rrbracket_{\text{er}} \triangleq \emptyset$$

from

$$\llbracket c \rrbracket \subseteq \Sigma \times \Sigma$$

to

$$\llbracket c \rrbracket_{\text{ok}} \subseteq \Sigma \times \Sigma$$

$$\llbracket c \rrbracket_{\text{er}} \subseteq \Sigma \times \Sigma$$

Semantics: compositions

$$\llbracket c_1 + c_2 \rrbracket \epsilon \triangleq \llbracket c_1 \rrbracket \epsilon \cup \llbracket c_2 \rrbracket \epsilon$$

$$\llbracket c^\star \rrbracket \epsilon \triangleq \bigcup_{k \in \mathbb{N}} \llbracket c^k \rrbracket \epsilon \quad \text{where } c^k \triangleq \underbrace{c; \cdots; c}_{k \text{ times}}$$

$$\llbracket c_1; c_2 \rrbracket \text{ok} \triangleq \llbracket c_2 \rrbracket \text{ok} \circ \llbracket c_1 \rrbracket \text{ok}$$

$$\llbracket c_1; c_2 \rrbracket \text{er} \triangleq \llbracket c_1 \rrbracket \text{er} \cup (\llbracket c_2 \rrbracket \text{er} \circ \llbracket c_1 \rrbracket \text{ok})$$

c_1 fails

c_1 ok, but
 c_2 fails

c_1 ok,
but ...

$$S, T \subseteq \Sigma \times \Sigma$$

$$T \circ S \triangleq \{(\sigma_1, \sigma_2) \mid \exists \sigma. (\sigma_1, \sigma) \in S \wedge (\sigma, \sigma_2) \in T\} \subseteq \Sigma \times \Sigma$$

$$\llbracket c \rrbracket \text{ok} \subseteq \Sigma \times \Sigma$$

$$\llbracket c \rrbracket \text{er} \subseteq \Sigma \times \Sigma$$

$$\llbracket c \rrbracket \epsilon \subseteq \Sigma \times \Sigma$$

any of
ok, er

Shorthand notation

$$[P] \text{ } c \text{ } [\text{ok} : Q_1][\text{er} : Q_2]$$

stands for

$$[P] \text{ } c \text{ } [\text{ok} : Q_1] \text{ and } [P] \text{ } c \text{ } [\text{er} : Q_2]$$

whose validity conditions are

$$Q_1 \subseteq \llbracket c \rrbracket_{\text{ok}} P \text{ and } Q_2 \subseteq \llbracket c \rrbracket_{\text{er}} P$$

Real IL rules

Atomic commands

$$[P] \ x := a \ [\text{ok} : \exists x'. P[x'/x] \wedge x = a[x'/x]] [\text{er} : \text{false}] \quad [\text{Floyd}]$$
$$[\text{skip}] \quad \frac{}{[P] \ \text{skip} \ [\text{ok} : P] [\text{er} : \text{false}]}$$

$$[P] \ b? \ [\text{ok} : P \wedge b] [\text{er} : \text{false}] \quad [\text{assume}]$$
$$[\text{error}] \quad \frac{}{[P] \ \text{error}() \ [\text{ok} : \text{false}] [\text{er} : P]}$$

$$[P] \ x := \text{nondet}() \ [\text{ok} : \exists x. P] [\text{er} : \text{false}] \quad [\text{nondet}]$$

Main
novelty!

Real atomic commands

...

[skip]

$[P]$ skip $[ok : P]$

[skip]

$[P]$ skip $[er : false]$

...

Sequence

[seq]

$$[P] \ c_1 \ [\text{ok} : R] \quad [R] \ c_2 \ [\epsilon : Q]$$

$$[P] \ c_1 ; c_2 \ [\epsilon : Q]$$

[seq_sc]

$$[P] \ c_1 \ [\text{er} : Q]$$

$$[P] \ c_1 ; c_2 \ [\text{er} : Q]$$

Short circuiting of errors!

$$[y = v] \ \text{error}() ; x := y \ [\text{er} : y = v]$$

$$[y = v] \ x := y ; \text{error}() \ [\text{er} : x = y = v]$$

Choice: dropping disjuncts

[choiceL]

$$[P] \ c_1 \ [\epsilon : Q]$$

$$[P] \ c_1 + c_2 \ [\epsilon : Q]$$

[choiceR]

$$[P] \ c_2 \ [\epsilon : Q]$$

$$[P] \ c_1 + c_2 \ [\epsilon : Q]$$

sound under-approximation!
scalable bug detection

$$[y = v] \text{ error}() + x := y \ [\text{er} : y = v]$$
$$[y = v] \text{ error}() + x := y \ [\text{ok} : x = y = v]$$

Example

$[y = x]$ if $even(x)$ then error() $[er : y = 42]$ 

is it a valid IL triple?

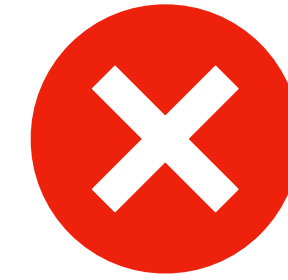
$(y = 42) \triangleq \{[x \mapsto 0, y \mapsto 42], [x \mapsto 1, y \mapsto 42], [x \mapsto 2, y \mapsto 42], \dots\}$


All the states above satisfy the post $[y = 42]$ since it does not constrain x in any way.
In particular, the condition is satisfied by states like δ below

$\delta \triangleq [x \mapsto 1, y \mapsto 42] \models (y = 42)$

But δ is unreachable! Therefore the triple is not valid

Example



$[y = x]$ if $even(x)$ then error() $[er : y = 42 \wedge even(x)]$

is it a valid IL triple?

$(y = 42) \triangleq \{ [x \mapsto 0, y \mapsto 42], [x \mapsto 2, y \mapsto 42], \dots, [x \mapsto 42, y \mapsto 42], \dots \}$



All the states above satisfy the post $[y = 42]$ since it does not constrain x in any way.
In particular, the condition is satisfied by states like δ below

$\delta \triangleq [x \mapsto 0, y \mapsto 42] \models (y = 42)$

But δ is unreachable! Therefore the triple is not valid

Example

$[y = x]$ if $even(x)$ then $error()$ $[er : x = y = 42]$ 

is it a valid IL triple?

$$(x = y = 42) \triangleq \{[x \mapsto 42, y \mapsto 42]\}$$


Dealing with division

arithmetic
expression

$$a ::= n \mid x \mid a_1 + a_2 \mid \dots \mid a_1 / a_2$$

$$\text{def}(n) \triangleq \text{true}$$

$$\text{def}(x) \triangleq \text{true}$$

$$\text{def}(a_1 + a_2) \triangleq \text{def}(a_1) \wedge \text{def}(a_2)$$

...

$$\text{def}(a_1 / a_2) \triangleq \text{def}(a_1) \wedge \text{def}(a_2) \wedge a_2 \neq 0$$

Dealing with division

boolean
expression

$$b ::= a_1 \leq a_2 \mid b_1 \wedge b_2 \mid \dots$$

...

$$\text{def}(a_1 \leq a_2) \triangleq \text{def}(a_1) \wedge \text{def}(a_2)$$

...

$$\text{def}(b_1 \wedge b_2) \triangleq \text{def}(b_1) \wedge \text{def}(b_2)$$

Atomic commands

$$[P] x := a \text{ [ok : } \exists x'. P[x'/x] \wedge \text{def}(a[x'/x]) \wedge x = a[x'/x] \text{]}$$

$$[P] x := a \text{ [er : } P \wedge \neg \text{def}(a) \text{]}$$

$$[P] b? \text{ [ok : } P \wedge \text{def}(b) \wedge b \text{]}$$

$$[P] b? \text{ [er : } P \wedge \neg \text{def}(b) \text{]}$$

Example 1

$[y = 100x]$
if $(y/x \geq 10)$ then
 $[\dots]$
 $y := y - x$
 $[\dots]$
else
 $[\dots]$
 $y := x$
 $[\dots]$
 $[\text{ok} : \dots]$

$[y = 100x]$
 $(y/x \geq 10)? ;$
 $[\dots]$
 $y := y - x$
 $[\dots]$
+ $(y/x < 10)? ;$
 $[\dots]$
 $y := x$
 $[\dots]$
 $[\text{ok} : \dots]$

Example 1

$[y = 100x]$
if $(y/x \geq 10)$ then
 $[\dots]$
 $y := y - x$
 $[\dots]$
else
 $[\dots]$
 $y := x$
 $[\dots]$
 $[\text{ok} : \dots]$

$[y = 100x]$
 $(y/x \geq 10)? ;$
 $[y = 100x \wedge x \neq 0]$
 $y := y - x$
 $[y = 99x \wedge x \neq 0]$
+ $(y/x < 10)? ;$
 $[\text{false}]$
 $y := x$
 $[\text{false}]$
 $[\text{ok} : y = 99x \wedge x \neq 0]$

Example 1

$[y = 100x]$
if $(y/x \geq 10)$ then
 $[\dots]$
 $y := y - x$
 $[\dots]$
else
 $[\dots]$
 $y := x$
 $[\dots]$
 $[\text{er} : \dots]$

$[y = 100x]$
 $(y/x \geq 10)? ;$
 $[\dots]$
 $y := y - x$
 $[\dots]$
+ $(y/x < 10)? ;$
 $[\dots]$
 $y := x$
 $[\dots]$
 $[\text{er} : \dots]$

Example 1

$[y = 100x]$
if $(y/x \geq 10)$ then
 $[\dots]$
 $y := y - x$
 $[\dots]$
else
 $[\dots]$
 $y := x$
 $[\dots]$
 $[\text{er} : \dots]$

$[y = 100x]$
 $(y/x \geq 10)? ;$
 $[\text{er} : y = x = 0]$
 $y := y - x$
 $[\text{er} : y = x = 0]$
+ $(y/x < 10)? ;$
 $[\text{er} : y = x = 0]$
 $y := x$
 $[\text{er} : y = x = 0]$
 $[\text{er} : y = x = 0]$

Example 2

$[y = i]$
while ($i > 0$) do
 $[...]$
 $y := y/2 ;$
 $[...]$
 $i := i/y$
 $[...]$
 $[er : ...]$

$[y = i]$
(($i > 0$)? ;
 $[...]$
 $y := y/2 ;$
 $[...]$
 $i := i/y)^* ;$
 $[...]$
 ($i < 0$)?
 $[...]$
 $[er : ...]$

Example 2

$[y = i]$
while $(i > 0)$ do
 $[...]$
 $y := y/2 ;$
 $[...]$
 $i := i/y$
 $[...]$
 $[er : ...]$

$[y = i]$
 $((i > 0)? ;$
 $[y = i > 0]$
 $y := y/2 ;$
 $[i > 0 \wedge y = i/2]$
 $i := i/y)^* ;$
 $[er : y = 0 \wedge i = 1]$
 $(i < 0)?$
 $[er : y = 0 \wedge i = 1]$
 $[er : y = 0 \wedge i = 1]$

Example 2

$[y = i]$
while ($i > 0$) do
 $[...]$
 $y := y/2 ;$
 $[...]$
 $i := i/y$
 $[...]$
 $[ok : ...]$

$[y = i]$
(($i > 0$)? ;
 $[...]$
 $y := y/2 ;$
 $[...]$
 $i := i/y)^* ;$
 $[...]$
 ($i < 0$)?
 $[...]$
 $[ok : ...]$

Example 2

$[y = i]$
while ($i > 0$) do
 $[...]$
 $y := y/2 ;$
 $[...]$
 $i := i/y$
 $[...]$
 $[ok : ...]$

$[y = i]$
(($i > 0$)? ;
 $[...]$
 $y := y/2 ;$
 $[...]$
 $i := i/y)^* ;$
 $[...]$
 ($i < 0$)?
 $[...]$
 $[ok : y = i \leq 0]$

Procedural abstraction

Over-approximation

```
void foo() {  
    % presumes  $\{x \geq 0\}$   
    ... % e.g.  $x := x + 1$   
    % achieves  $\{x > 0\}$   
}  
  
void client() {  
    % presumes  $\{x \geq 0\}$   
    foo();  
    %  $\{x > 0\} \Rightarrow \{x \geq 0\}$   
    foo()  
    % achieves  $\{x > 0\}$   
}
```

```
void test() {  
     $x := 0$ ;  
    client();  
    if ( $x < 2$ ) then error()  
    % do we have a warning?  
}
```


Procedural abstraction

Over-approximation

```
void foo() {  
    % presumes  $\{x \geq 0\}$   
    ... % e.g.  $x := x + 1$   
    % achieves  $\{x > 0\}$   
}  
  
void client() {  
    % presumes  $\{x \geq 0\}$   
    foo();  
    %  $\{x > 0\} \Rightarrow \{x \geq 0\}$   
    foo()  
    % achieves  $\{x > 0\}$   
}
```

```
void test() {  
    {true}  
     $x := 0$ ;  
     $\{x = 0\} \Rightarrow \{x \geq 0\}$   
    client();  
     $\{x > 0\}$   
    if ( $x < 2$ ) then error()  
    % do we have a warning?  
    % yes:  $x = 1$   
}
```

Procedural abstraction

Under-approximation

```
void foo() {  
    % presumes  $[x \geq 0]$   
    ... % e.g.  $x := x + 1$   
    % achieves  $[x > 0]$   
}
```

```
void client() {  
    % presumes  $[x \geq 0]$   
    foo();
```

~~$[x > 0] \Rightarrow [x \geq 0]$~~ % under-approximations cannot be weakened!

```
    foo()  
    % achieves: nothing known!  
}
```

```
void test() {  
     $x := 0$  ;  
    client() ;  
    if ( $x < 2$ ) then error()  
    % do we have a warning?  
    % inconclusive...  
}
```


Frame

assigned variables in c
are disjoint from
free variables in R

$$\frac{[P] \ c \ [Q]}{[P \wedge R] \ c \ [Q \wedge R]} \text{ [frame]}$$

$$[x \neq y] \ x := y; \text{error}() \ [\text{er} : x = y]$$

$$[x \neq y \wedge y = 42] \ x := y; \text{error}() \ [\text{er} : x = y = 42]$$

Ghost variables

ghost variable

$$[x = n \wedge n \geq 0] \text{foo}() [x = n + 1 \wedge n \geq 0]$$

n can be $m + 1$

$$[x = m + 1 \wedge m + 1 \geq 0] \text{foo}() [x = m + 2 \wedge m + 1 \geq 0]$$

$$[x = m + 1 \wedge m \geq 0] \text{foo}() [x = m + 2 \wedge m \geq 0]$$

framing with
 $m \geq 0$

framing with
 $m \geq 0$

Ghost variables + Frame

```
void foo() {  
    % presumes  $[x = n \wedge n \geq 0]$   
    %  $[x = m + 1 \wedge m \geq 0]$   
    ... % e.g.  $x := x + 1$   
    % achieves  $[x = n + 1 \wedge n \geq 0]$   
    %  $[x = m + 2 \wedge m \geq 0]$   
}  
void client() {  
    % presumes  $[x = n \wedge n \geq 0]$   
    foo();  
    %  $[x = n + 1 \wedge n \geq 0]$   
    foo()  
    % achieves:  $[x = n + 2 \wedge n \geq 0]$   
}
```

```
void test() {  
    [true]  
     $x := 0$ ;  
     $[x = 0] \equiv [x = n \wedge n \geq 0 \wedge n = 0]$   
    client();  
     $[x = n + 2 \wedge n \geq 0 \wedge n = 0]$   
    if ( $x < 3$ ) then error()  
    % do we have a warning?  
    % yes:  $er : x = 2$   
}
```


Valid presumptions need not exist

HL: Given c and Q , there is a (weakest)
precondition P such that $\{P\} c \{Q\}$
proof: take $P = wlp(c, Q)$

IL: Given c and Q , a (weakest) precondition P
such that $[P] c [Q]$ may not exist
proof: take Q with some unreachable state
e.g. $[?] x := 42 [true]$

Exercises

One from lecture #04

Find a derivation for the HL triple

$\{\text{true}\}$ if $x \geq y$ then $z := x$ else $z := y$ $\{z = \max(x, y)\}$

$\{\text{true}\}$

if $x \geq y$ then

$\{\dots\}$

$z := x$

$\{\dots\}$

else

$\{\dots\}$

$z := y$

$\{\dots\}$

$\{z = \max(x, y)\}$

One from lecture #04

Find a derivation for the HL triple

$\{\text{true}\}$ if $x \geq y$ then $z := x$ else $z := y$ $\{z = \max(x, y)\}$

$\{\text{true}\}$

if $x \geq y$ then

$\{x \geq y\}$

$z := x$

$\{z = x \geq y\} \Rightarrow \{z = \max(x, y)\}$

else

$\{y > x\}$

$z := y$

$\{z = y > x\} \Rightarrow \{z = \max(x, y)\}$

$\{z = \max(x, y)\}$

Another one from lecture #04

Prove that rule {conj} is sound

$$\frac{\{P_1\} \text{ c } \{Q_1\} \quad \{P_2\} \text{ c } \{Q_2\}}{\{P_1 \wedge P_2\} \text{ c } \{Q_1 \wedge Q_2\}} \text{ {conj} }$$

Another one from lecture #04

Prove that rule {conj} is sound
$$\frac{\{P_1\} c \{Q_1\} \quad \{P_2\} c \{Q_2\}}{\{P_1 \wedge P_2\} c \{Q_1 \wedge Q_2\}} \text{ {conj} }$$

Assume $\llbracket c \rrbracket P_1 \subseteq Q_1$ and $\llbracket c \rrbracket P_2 \subseteq Q_2$

By monotonicity of $\llbracket c \rrbracket$ we have:

$\llbracket c \rrbracket (P_1 \wedge P_2) \subseteq \llbracket c \rrbracket P_1 \subseteq Q_1$ and

$\llbracket c \rrbracket (P_1 \wedge P_2) \subseteq \llbracket c \rrbracket P_2 \subseteq Q_2$

Therefore $\llbracket c \rrbracket (P_1 \wedge P_2) \subseteq Q_1 \wedge Q_2$

One more from lecture #04

Show that the following rule for assignment is **not sound**

$$\frac{}{\{P\} x := a \{P[a/x]\}}$$

syntax
replacement

One more from lecture #04

Show that the following rule for assignment is **not sound**

$$\frac{}{\{P\} x := a \{P[a/x]\}}$$

syntax
replacement

Consider the instance $\{y = x\} x := 0 \{y = 0\}$

then $\llbracket x := 0 \rrbracket [x \mapsto 1, y \mapsto 1] = [x \mapsto 0, y \mapsto 1] \not\models \{y = 0\}$

Validity

Which triples are valid for any c and R ?

$[R] \ c \ [\text{ok} : \text{false}][\text{er} : \text{false}]$ ✓

$\{R\} \ c \ \{\text{false}\}$ ✗

$[R] \ c \ [\text{ok} : \text{true}]$ ✗

$\{R\} \ c \ \{\text{true}\}$ ✓

$[\text{true}] \ c \ [\text{ok} : R]$ ✗

$\{\text{false}\} \ c \ \{R\}$ ✓

$[wlp(c, R)] \ c \ [\text{ok} : R]$ ✗

$\{wlp(c, R)\} \ c \ \{R\}$ ✓

Derivation

Find a derivation for the IL triple

[true] if $x \geq y$ then $z := x$ else $z := y$ **[ok : $z = \max(x, y)$]**

[true]

if $x \geq y$ then

$z := x$

else

$z := y$

Derivation

Find a derivation for the IL triple

$[\text{true}]$ if $x \geq y$ then $z := x$ else $z := y$ $[\text{ok} : z = \max(x, y)]$

$[\text{true}]$

if $x \geq y$ then

$[x \geq y]$

$z := x$

$[z = x \geq y] \equiv [x \geq y, z = \max(x, y)]$

else

$[y > x]$

$z := y$

$[z = y > x] \equiv [y > x, z = \max(x, y)]$

$[\text{ok} : z = \max(x, y)]$

$\{\text{true}\}$

if $x \geq y$ then

$\{x \geq y\}$

$z := x$

$\{z = x \geq y\} \Rightarrow \{z = \max(x, y)\}$

else

$\{y > x\}$

$z := y$

$\{z = y > x\} \Rightarrow \{z = \max(x, y)\}$

$\{z = \max(x, y)\}$

Soundness

Prove that rule [conj] is **not sound**

$$\frac{[P_1] \text{ c } [\epsilon : Q_1] \quad [P_2] \text{ c } [\epsilon : Q_2]}{[P_1 \wedge P_2] \text{ c } [\epsilon : Q_1 \wedge Q_2]} \text{ [conj]}$$

Soundness

Prove that rule [conj] is **not sound**

$$\frac{[P_1] \text{ } c \text{ } [\epsilon : Q_1] \quad [P_2] \text{ } c \text{ } [\epsilon : Q_2]}{[P_1 \wedge P_2] \text{ } c \text{ } [\epsilon : Q_1 \wedge Q_2]} \text{ [conj]}$$

We need to find a counterexample.

Take $c \triangleq x := 7$, $P_1 \triangleq (x = 0)$, $P_2 \triangleq (x = 1)$, $Q_i \triangleq (x = 7)$

Both the triples in the premises are valid, but with

$P_1 \wedge P_2 \equiv \text{false}$, $Q_1 \wedge Q_2 \triangleq (x = 7)$

the triple $[false] \text{ } x := 7 \text{ } [ok : x = 7]$ is **not valid**

because the premise is false and $\llbracket x := 7 \rrbracket false \not\subseteq (x = 7)$.

Soundness

Show that the following rule for assignment is **not sound**

$$\frac{}{[P] x := a \text{ [ok : } P[a/x]\text{]}}$$

syntax
replacement

Soundness

Show that the following rule for assignment is **not sound**

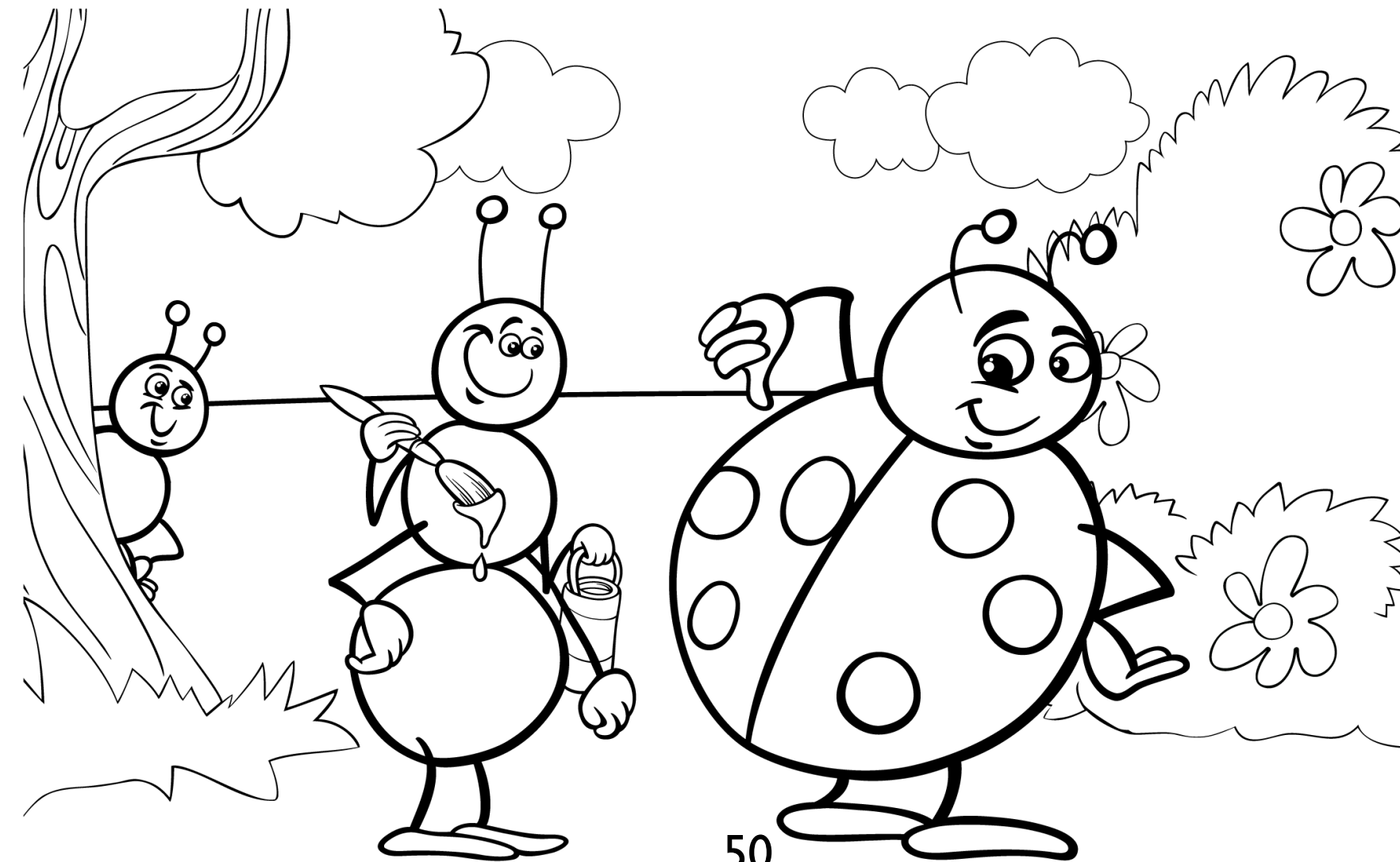
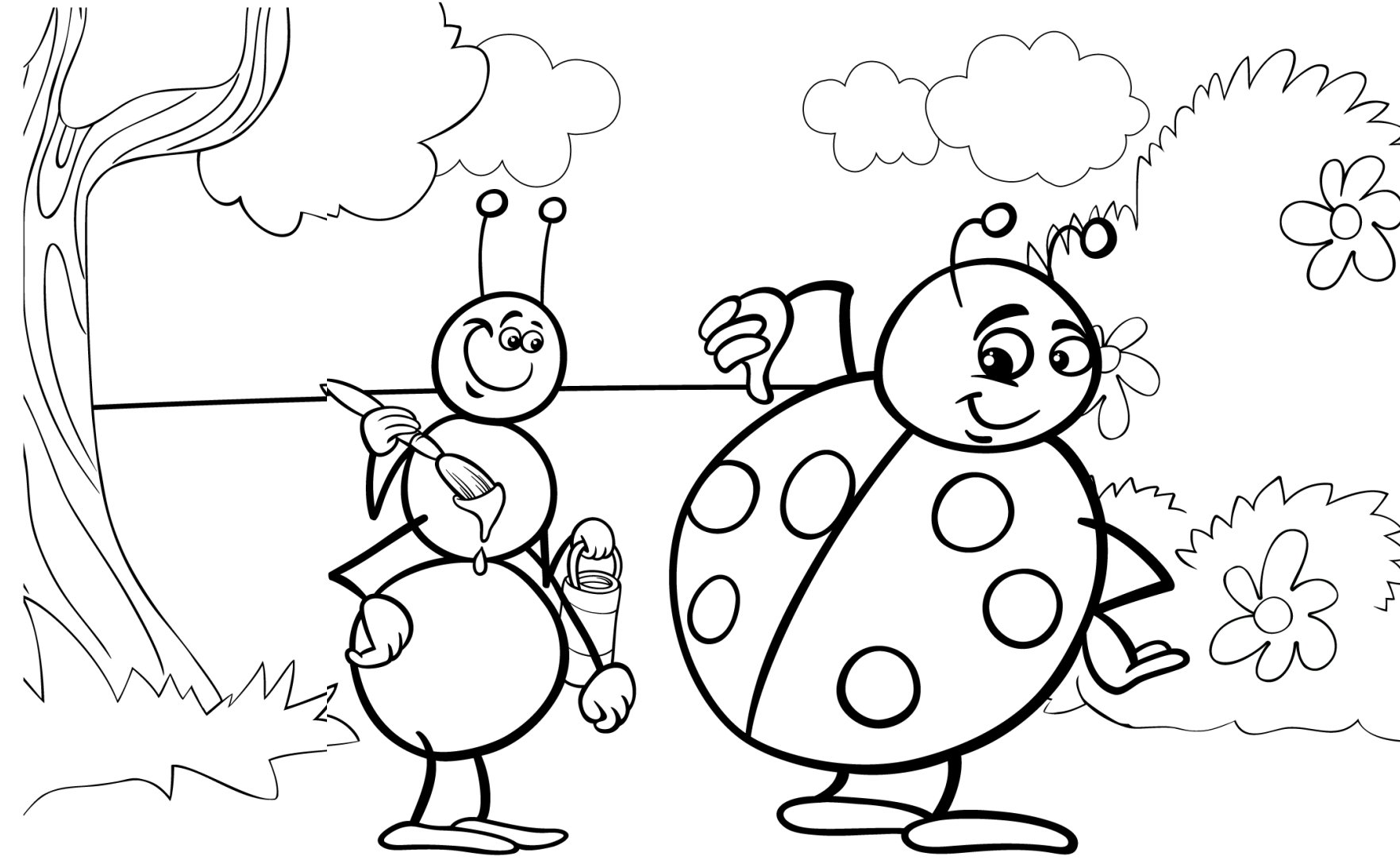
$$\frac{}{[P] \ x := a \ [\text{ok} : P[a/x]]}$$

syntax
replacement

Consider the instance $[x = y] \ x := 0 \ [\text{ok} : y = 0]$

then $(x \mapsto 1, y \mapsto 0) \models (y = 0)$ but is not a reachable state!

Find the difference (bug)



Find the typo (bug)

Empty under-approximates

$$[p]C[\epsilon: false]$$

Unit

$$[p]skip[ok:p][er:false]$$

Iterate zero

$$[p]C^*[ok:p]$$

Choice (where $i = 1$ or 2)

$$\frac{[p]C_i[\epsilon: q]}{[p]C_1 + C_2[\epsilon: q]}$$

Assignment

$$[p]x = e[ok: \exists x'. p[x'/x] \wedge x = e[x'/x]][er:false]$$

Constancy

$$\frac{[p]C[\epsilon: q]}{[p \wedge f]C[\epsilon: q \wedge f]} \quad Mod(C) \cap Free(f) = \emptyset$$

Substitution I

$$\frac{[p]C[\epsilon: q]}{([p]C[\epsilon: q])(e/x)} \quad (Free(e) \cup \{x\}) \cap Free(C) = \emptyset$$

Consequence

$$\frac{p' \Leftarrow p \quad [p]C[\epsilon: q] \quad q \Leftarrow q'}{[p']C[\epsilon: q']}$$

Sequencing (short-circuit)

$$\frac{[p]C_1[er:r]}{[p]C_1; C_2[er:r]}$$

Iterate non-zero

$$\frac{[p]C^*; C[\epsilon: q]}{[p]C^*[\epsilon: q]}$$

Error

$$[p]error()[ok:false][er:p]$$

Disjunction

$$\frac{[p_1]C[\epsilon: q_1] \quad [p_2]C[\epsilon: q_2]}{[p_1 \vee p_2]C[\epsilon: q_1 \vee q_2]}$$

Sequencing (normal)

$$\frac{[p]C_1[ok:q] \quad [q]C_2[\epsilon: r]}{[p]C_1; C_2[\epsilon: r]}$$

Backwards Variant (where n fresh)

$$\frac{[p(n) \wedge nat(n)]C[ok:p(n+1) \wedge nat(n)]}{[p(0)]C^*[ok: \exists n. p(n) \wedge nat(n)]}$$

Assume

$$[p]assume B[ok:p \wedge B][er:false]$$

Nondet Assignment

$$[p]x = nondet()[ok: \exists x'p][er:false]$$

Local Variable

$$\frac{[p]C(y/x)[\epsilon: q]}{[p]local x.C[\epsilon: \exists y.q]} \quad y \notin Free(p, C)$$

Substitution II

$$\frac{[p]C[\epsilon: q]}{([p]C[\epsilon: q])(y/x)} \quad y \notin Free(p, C, q)$$